When Generated Answers Become Evidence: Do Errors Compound in Self-Updating Retrieval-Augmented Generation?

Pavan Kumar Guthikonda
Independent researcher (M.Tech alumnus, BITS Pilani)
pavansky143@gmail.com

Draft — October 2026

Abstract

A self-updating support system stores the answers its users accept, so that the next similar question can be answered from them. Our previous paper measured this loop when the stored entry is a pointer to an existing thread and no text is generated [Guthikonda, 2026]. In deployed retrieval-augmented generation (RAG) systems the stored entry is a generated answer, and a retrieved entry becomes the evidence from which the next answer is generated. We measure that loop on the same real data (CQADupStack forums, community duplicate links as ground truth, real repeat questions replayed through bge-small, MiniLM and BM25), with answers generated by a fixed model and only user acceptance simulated. We test four hypotheses stated before the runs. (H1) Stored generated answers are retrieved in place of the correct human-written thread more often than stored thread text is: *not supported*. Entries that store the question with the generated answer are the safest design we have measured on these forums, with no crossover below $p_w = 0.95$ in 8 of 12 forum–retriever configurations, against 3 for the question-only entries of the previous paper, and a median steal rate of 3% against 13%. (H2) Errors compound: *weakly*. Entries generated from entries appear (5–13% of entries at $p_w = 0.95$), and each level moves the answer slightly further from the thread's accepted answer, but they account for only 0.3–2.5% of wrong answers against 4.7–11.7% for first-level entries. (H3) A depth-aware correction to the plug-in estimate of p^* is *not needed*: the first-order estimate is as accurate (mean absolute error 0.122 against 0.141). (H4) Reopen demotion is the best countermeasure: it keeps the full gain at low p_w and raises the mean gain at $p_w = 0.95$ from +0.07 to +0.19 points, while provenance down-weighting removes most of the gain and storing the cited thread is inconsistent. For practitioners, the risk in a human-gated loop sits at the first level, where a wrongly accepted answer is retrieved and copied once, and a cheap reopen signal addresses it.

1 Introduction

Retrieval-augmented support systems increasingly learn from their own use: when a user accepts an answer, the question and the answer are written back into the knowledge base, and later questions can retrieve that entry [Liu et al., 2026, Wang et al., 2026a, Guan et al., 2026, Consortium for Service Innovation, 2016]. Our previous paper studied the retrieval half of that loop [Guthikonda, 2026]: the stored entry pointed to an existing human-written thread, so a wrong entry could misdirect retrieval but could not change the text of any answer. It found a crossover acceptance rate of wrong answers, p^*, beyond which write-back hurts, showed that the harm comes from question-to-question similarity, and predicted p^* from an exact per-entry decomposition.

Deployed systems do not store pointers. They store the answer the model generated, and when that entry is retrieved, its text is the evidence the model generates from next. Two things change. First, the stored text is model-written, and neural retrievers prefer model-written text [Dai et al., 2024, Chen et al., 2024], so a stored generated answer may be retrieved in place of the human-written thread even when the thread is the better match. Second, a wrong stored answer can become the source of the next generated answer, which can be stored in turn: the error is no longer first order. Related work on generated text in corpora treats the pollution as external and un-gated [Yu et al., 2026b, Druck and Smith, 2026]; work on agent memory shows that self-written experience propagates errors [Xiong et al., 2026, Shao et al., 2026]. We measure the closed, human-gated case on real repeat questions, with the same corpus, retrievers, ground truth and protocol as before, so that every number is directly comparable with the pointer-only loop.

The contributions are four tested hypotheses, each with its table:

1. **No extra source bias in the gated loop (H1).** Storing the question with the generated answer is safer than every pointer design of the previous paper. Generated text is retrieved, but it rarely displaces the correct thread.

2. **Compounding is real but small (H2).** Second- and third-generation entries form and drift slightly with each level, yet they cause a small fraction of wrong answers.

3. **The first-order model still predicts the crossover (H3).** A depth-aware correction does not improve the fit, because deep entries are too rare to change the average effect of a wrong entry.

4. **Reopen demotion is the countermeasure to deploy (H4).** It is the only rule that improves the high-p_w outcome without giving up the low-p_w gain.

2 Related work

Generated text in retrieval corpora. Dai et al. [2024] showed that neural retrievers rank model-written text above human-written text of the same content, and Chen et al. [2024] showed that, when generated answers are added to an open-domain QA corpus, they increasingly displace human documents over repeated rounds. Yu et al. [2026b] and Druck and Smith [2026] study the same effect at web scale and inside RAG, where self-authored documents crowd out the rest, and Huang et al. [2026] propose detecting generated text before it is indexed. In all of these the generated text enters the corpus from outside and un-gated: nothing checks it, and the document is not the evidence from which the next answer is written. Our loop differs on both counts. Every stored entry has passed a user's acceptance, and a retrieved entry is the evidence for the next generation, so the question is not only whether generated text is retrieved too often but whether that retrieval changes the next answer.

Agent memory that stores its own outputs. Experience memory for LLM agents writes the agent's own trajectories or summaries back for later reuse. Xiong et al. [2026] find that agents follow stored experience even when it is wrong, and Shao et al. [2026] and Xie et al. [2026] catalogue the risks of self-evolving memory, including error propagation. Hu et al. [2026] treat memory reuse as continual learning; Wang et al. [2026b] persist verifier signals with each memory, Yu et al. [2026a] repair memories by dependency-guided rollback, and Zhang et al. [2026] learn what to store. These are the closest in mechanism to our setting, since a stored item can be derived from an earlier stored item. They are evaluated on agent benchmarks with synthetic or model-judged success,

and the write policy is part of the agent. We measure the mechanism on real repeat questions with community ground truth, with the write gated by a human, and we track the depth of every stored item so that the contribution of second- and third-generation entries can be separated from first-generation ones.

The pointer-only loop. Guthikonda [2026] measured write-back on the same forums with entries that point to an existing thread. Three things carry over unchanged: the replay protocol, the crossover acceptance rate p^* as the summary statistic, and the per-entry decomposition that predicts p^* from the static accuracy and the average effect of a correct and a wrong entry. Two do not. A pointer entry cannot change any answer's text, so the loss was bounded by misdirected retrieval; here the stored text is model-written and a wrong entry can seed further wrong entries. And the previous decomposition is first order: every wrong entry costs the same, whatever its origin. Hypothesis H3 asks whether that still holds when entries have depth.

3 Setup

Data and retrievers. As in Guthikonda [2026]: CQADupStack forums [Hoogeveen et al., 2015] in BEIR form [Thakur et al., 2021], community duplicate links as ground truth, families of real repeat questions, three retrievers (bge-small-en-v1.5 [Xiao et al., 2024], all-MiniLM-L6-v2 [Wang et al., 2020, Reimers and Gurevych, 2019], BM25 [Robertson and Zaragoza, 2009]), ten random orders, ten rounds. This paper uses 4 forums (android, unix, webmasters, mathematica), the four on which the previous paper ran its cross-encoder experiments. Each base thread is completed with its accepted answer (or, failing that, its top-voted answer) fetched by post id from the public Stack Exchange API; threads with no recorded answer keep only the question text and are flagged.

The loop. For each incoming question the system retrieves the top-5 among base threads and stored entries, exactly as before. The top-1 item is the *evidence*; the system's answer is generated from it by a fixed model (`gpt-5.4-mini` on Azure AI Foundry, reasoning effort minimal, fixed seed, at most 120 words, with an instruction to use only the evidence). The few prompts that Azure's content filter rejects (1 of 31515) are generated by an open-weight model instead (`google/gemma-4-26b-a4b-qat`, 4-bit, temperature 0) and are marked in the released cache. The same open-weight model generates every answer for the android replication in Appendix A. The user accepts a correct answer with probability $p_c = 0.9$ and a wrong one with probability p_w, the swept parameter. An accepted answer is stored as an entry carrying the question, the generated answer, and a provenance chain: the evidence it was generated from, which is either a base thread (depth 1) or another entry (depth 2 and deeper). Correctness is evidence-based and transitive, as in the previous paper: an answer is correct when the base thread at the root of its evidence chain is the community-linked duplicate. This keeps the label objective; an audit of whether generated answers are faithful to their evidence is in Appendix B. When an entry is the evidence, the generator sees the stored question and the stored generated answer, never the root thread.

What the retriever indexes. Three entry designs, mirroring the previous paper's ablation: the generated answer alone (*gen*); the question together with the generated answer (*ask+gen*, the realistic self-written Q&A entry and our main condition); and the generated answer together with the text of the root thread (*gen+thread*, the "store with cited source" countermeasure). Entries are embedded by the deployed dense retriever or scored by BM25 with the base corpus statistics.

Every generated answer is produced once per (question, evidence) pair and cached, so all conditions, orders and retrievers that reach the same pair see the same text.

Conditions. Static (no write-back); naive write-back; reopen demotion (a wrongly accepted answer is reopened with probability 0.5 in the next round and its entry removed); and provenance down-weighting, which subtracts λ times the interquartile range of the static top-1 scores from every entry's score ($\lambda \in \{0.5, 1\}$, label-free and scale-free). Acceptance is the only simulated component. Every number is produced by the released scripts from saved runs.

4 Hypotheses and results

4.1 H1: source bias

Table 1 compares the three generated-entry designs with the two pointer designs of the previous paper on the same forums, retrievers, orders and acceptance model. Δ is the change in accuracy over the stream against static retrieval, in percentage points, and the steal rate is the share of questions whose correct thread was in the static top-1 and is displaced by an entry. If neural retrievers preferred model-written entries, the dense retrievers would show higher steal rates for the generated designs than for the stored-thread design, and BM25 would not.

They do not. The realistic entry, the question with the generated answer, has no crossover below $p_w = 0.95$ in 8 of 12 configurations; the previous paper's question-only entry has none in 3 and its question-plus-thread entry in 4. Median steal rates are 3% (maximum 6%) for question plus generated answer, 3% (maximum 7%) for question plus thread, and 13% (maximum 39%) for the question alone. The generated answer on its own (median 4%, maximum 11%) is the weakest of the three generated designs on bge, the retriever with the strongest question-to-question match, which is the same mechanism the previous paper identified: the harm comes from entries whose text resembles a new question more than the correct thread does, not from who wrote it. Adding the generated answer to the question dilutes that match in the same way that adding the thread did. We find no evidence of source bias in this loop. A likely reason is that our generated answers are short and paraphrase a single thread, so they carry little text that the thread itself does not.

4.2 H2: compounding

Every stored entry carries the chain of evidence it was generated from, so its depth is known. Table 2 reports, for the question-plus-answer design, the entries by depth at the end of the stream, the share of questions in the last round answered from an entry of depth 2 or more, the share of wrong answers whose evidence was a depth-1 or a deeper entry, and drift: the bge similarity between the served answer and the root thread's accepted answer, by depth of the evidence.

Chains form. At $p_w = 0.95$, 5–13% of entries are generated from another entry, and up to 4.9% of last-round answers come from such an entry. Each level moves the answer slightly away from the human answer: mean similarity is 0.85 when the evidence is a thread, 0.82 at depth 1 and 0.80 at depth 2 or more. The faithfulness audit (Appendix B) shows the same gradient in the text: the share of answers that no longer state the root thread's solution rises from 12–13% when the evidence is a thread to 13–18% at depth 1 and 21–23% at depth 2 or more, depending on the judge. What does not happen is a cascade. Deeper entries account for 0.3–2.5% of wrong answers, against 4.7–11.7% for depth-1 entries; the rest come from wrong static retrieval. A depth-2 entry needs a wrongly accepted depth-1 entry to be retrieved, served, and wrongly accepted again, and each step

is filtered by retrieval and by the user. Compounding exists in this loop, but it is a second-order effect.

4.3 H3: a depth-aware crossover estimate

The previous paper predicted p^* from the static accuracy and the average effect of a correct and of a wrong entry, measured at a single reference $p_w = 0.6$. If deeper entries made wrong write-backs more costly, that first-order estimate would sit above the observed crossover. The depth-aware variant splits the wrong-entry effect by depth and scales it by the observed exposure of deeper entries. Over the 15 configuration–design pairs with an observed crossover, the first-order estimate has mean absolute error 0.122 and the depth-aware estimate 0.141 (Table 3). The correction moves the estimate down, as designed, but it over-corrects: deep entries are rare enough that the first-order average already contains them. The estimate is weakest on BM25 for the generated-answer design on unix and on the mathematica configurations, where accuracy is low and the curve is flat, so a small change in Δ moves the crossover far.

4.4 H4: countermeasures

A countermeasure is worth deploying if it removes loss at high p_w without removing the gain at low p_w. Table 4 reports Δ at $p_w = 0.1$, 0.6 and 0.95 and the crossover for naive write-back and the three rules.

Reopen demotion, which removes an entry when a wrongly accepted answer is reopened with probability 0.5 in the next round, keeps the mean gain at $p_w = 0.1$ (0.29 points against 0.29) and raises the mean outcome at $p_w = 0.95$ from +0.07 to +0.19 points. It is better than naive write-back at $p_w = 0.95$ in 11 of 12 configurations and has no crossover below 0.95 in 9, against 8. Provenance down-weighting is safe by construction (no crossover in 10 configurations at $\lambda = 1$) because it hides entries: the mean gain at $p_w = 0.1$ falls to 0.07 points at $\lambda = 0.5$, so it buys safety by giving up the reason to write back. Storing the generated answer with its cited thread helps on some configurations and hurts on others (mean +0.06 at $p_w = 0.95$, no crossover in 7): adding the thread's text makes the entry a better match for questions about that thread, which helps when the thread is right and amplifies the damage when it is wrong. The ranking is reopen demotion first, then naive write-back, then storing the cited thread, with provenance down-weighting as the conservative option when a reopen signal is unavailable.

5 Limitations

Acceptance is simulated, as in the previous paper: a real user's probability of accepting a wrong answer depends on the answer, and wrong answers that read well may be accepted more often than our constant p_w assumes. We use one generator, one prompt and a fixed length limit; a generator that writes longer, more confident or more generic answers could produce entries that match more questions and so behave more like the question-only entries of the previous paper. The open-weight replication on android (Appendix A) is a first check on this. Correctness is evidence-based and transitive, which ignores answers that are right despite wrong evidence or wrong despite right evidence; the audit shows that most answers track their evidence, but a growing minority at depth 2 or more no longer states the root thread's solution, and those answers are labelled by their root rather than by what they say. Answers were fetched from the Stack Exchange API in 2026 for a 2014 snapshot, so some accepted answers have been edited since. Four forums with low static accuracy (9–23% top-1) limit how much any entry can help; the gains we report are small in

absolute terms. Finally, our entries are short and the base corpus is human-written; a deployment whose knowledge base is itself mostly generated may behave differently.

6 Conclusion

Writing generated answers back into a support knowledge base, behind a human acceptance gate, does not create the runaway feedback that work on un-gated generated text might suggest. On real repeat questions from four forums, the realistic entry design is safer than the pointer designs we measured before, retrievers do not prefer it over the correct thread, and entries generated from entries form but stay rare and cause little of the harm. The first-order model of the previous paper still predicts where write-back stops paying off. The risk that remains is the first-level one: a wrongly accepted answer that is retrieved and copied for the next similar question. Reopen demotion, which only needs a signal that an accepted answer did not hold, removes most of that risk without giving up the gain, and we recommend it as the default guard. Every number in this paper is produced by the released scripts from the released cache of 31515 generated answers.

Code and data availability

All data are public (CQADupStack via BEIR/MTEB; answers via the Stack Exchange API, cached and released). Code, generated answers and the scripts that produce every table are at `https://github.com/pavansky/self-updating-retrieval` (paper 2 files: `prep_gen.py`, `sim_gen.py`, `analyze_gen.py`, `audit_gen.py`, `run_gen.sh`, `run_gen_az.sh`; generated answers and judgements in `data_gen/`). Stack Exchange content is redistributed under CC BY-SA 4.0.

Acknowledgements

This work uses no employer or customer data. The retrieval setup follows TicketIQ, the author's M.Tech dissertation system.

References

Xiaoyang Chen, Ben He, Hongyu Lin, Xianpei Han, Tianshu Wang, Boxi Cao, Le Sun, and Yingfei Sun. Spiral of silence: How is large language model killing information retrieval?—A case study on open domain question answering. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14930–14951, 2024. doi: 10.18653/v1/2024.acl-long.798.

Consortium for Service Innovation. KCS v6 practices guide. `https://library.serviceinnovation.org/KCS/KCS_v6/KCS_v6_Practices_Guide`, 2016. Knowledge-Centered Service (KCS) v6, released 21 April 2016.

Sunhao Dai, Yuqi Zhou, Liang Pang, Weihao Liu, Xiaolin Hu, Yong Liu, Xiao Zhang, Gang Wang, and Jun Xu. Neural retrievers are biased towards LLM-generated content. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024. doi: 10.1145/3637528.3671882. arXiv:2310.20501.

Gregory Druck and Ethan Smith. RAG collapse: LLM responses collapse when retrieved documents are self-authored, 2026. arXiv:2608.22118.

Boyuan Guan, Hailu Xu, and Jamie Rogers. Leveraging resolved incident history for LLM-assisted software bug diagnosis. In *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2026. URL `https://arxiv.org/abs/2607.21911`. Accepted; arXiv:2607.21911.

Pavan Kumar Guthikonda. When accepted answers become evidence: Feedback loops in self-updating retrieval. Zenodo preprint, version 3, 2026. All versions: doi:10.5281/zenodo.23089817.

Doris Hoogeveen, Karin M. Verspoor, and Timothy Baldwin. CQADupStack: A benchmark data set for community question-answering research. In *Proceedings of the 20th Australasian Document Computing Symposium (ADCS)*, pages 3:1–3:8. ACM, 2015.

Qisheng Hu, Quanyu Long, and Wenya Wang. When continual learning moves to memory: A study of experience reuse in LLM agents. arXiv preprint arXiv:2604.27003, 2026.

Zhaoheng Huang, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. LLM-generated text may harm your retrieval! a robust detection strategy for retrieval-augmented generation. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31973–31988, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.1475. URL `https://aclanthology.org/2026.acl-long.1475/`.

Fengrui Liu, Xiao He, and Tieying Zhang. Help without being asked: A deployed proactive agent system for on-call support with continuous self-improvement. arXiv preprint arXiv:2604.09579, 2026.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, pages 3982–3992, 2019.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.26354.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

Chih Hui Wang, Mengdie Tu, Qianyun Zhang, Wei Wu, Lili Zhou, Mingqi Shen, and Changshuai Wei. Self-evolving agentic customer support system at LinkedIn. arXiv preprint arXiv:2608.10224, 2026a.

Haoyu Wang, Guangyuan Dong, He Liang, Zijing Zhang, Jiachen Luo, Chuang Liu, Chao Xue, and Hao Tang. MemGuard: Persisting verifier signals for LLM-agent memory governance. arXiv preprint arXiv:2608.21867, 2026b.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33*, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi: 10.1145/3626772.3657878.

Weiwei Xie, Shaoxiong Guo, Fan Zhang, Tian Xia, Xue Yang, Lizhuang Ma, Junchi Yan, and Qibing Ren. MemEvoBench: Benchmarking safety risks from memory misevolution in LLM agents. arXiv preprint arXiv:2604.15774, 2026.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.27. URL `https://aclanthology.org/2026.acl-long.27/`. arXiv:2505.16067.

Caili Yu, Yiqi Wang, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, and Taotao Cai. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents. arXiv preprint arXiv:2608.10502, 2026a.

Hongyeon Yu, Dongchan Kim, and Young-Bum Kim. Retrieval collapses when AI pollutes the web. In *Proceedings of the ACM Web Conference 2026*, 2026b. arXiv:2602.16136.

Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory. arXiv preprint arXiv:2601.03192, 2026.

A Open-weight replication on android

To check that the results do not depend on the generator, we repeated the full android loop with the open-weight model `google/gemma-4-26b-a4b-qat` (4-bit, temperature 0, same prompt and length limit) generating every answer, run locally. The loop needed 9375 generated answers. Everything else is identical: retrievers, orders, acceptance model and conditions.

The conclusions hold. The question-plus-answer entry has no crossover below $p_w = 0.95$ in 3 of 3 android configurations (main generator: all three), and its median steal rate is 3%, against 10% for the previous paper's question-only entry (Table 5). Deep entries account for 0.5–2.3% of wrong answers at $p_w = 0.95$, against 6.5–12.0% for depth-1 entries, and drift is 0.85, 0.83 and 0.82 by depth (Table 6). The first-order plug-in estimate again fits at least as well as the depth-aware one (mean absolute error 0.038 against 0.106). The open-weight generator's answers drift slightly less from the thread at depth 1 and 2 than the main generator's, which is consistent with a model that paraphrases its evidence more closely.

B Faithfulness audit

Correctness in the main text follows the evidence chain to its root thread. This audit checks whether the generated text actually carries the root thread's solution, and how that changes with depth. We sampled 300 cached answers, 25 per forum at each evidence depth (thread, depth 1,

depth 2 or more), among keys whose root thread has an answer on record. Two judges read the question, the root thread's accepted (or top) answer and the generated answer, and replied YES (same solution, possibly less detailed), PARTLY or NO. One judge is the generator's own model, `gpt-5.4-mini`; the other is the open-weight model, which did not write these answers. They agree on 82% of answers. Most sampled answers have a wrong root, because top-1 retrieval on these forums is usually wrong; the audit measures faithfulness to the evidence, not accuracy.

Answers generated from a thread state its solution in most cases. Each level of indirection loses a little: the NO share rises from 12% (gpt-5.4-mini judge) and 13% (open-weight judge) at the thread to 21% and 23% at depth 2 or more. This is the textual counterpart of the drift in Table 2. It also bounds the error of the transitive label: at depth 2 or more about one answer in five is labelled by a root whose solution it no longer states. Because deep entries serve a small share of answers, this does not change the conclusions of Section 4. The sample of answers with a correct root is small (5 to 10 per depth), so we do not compare faithfulness between correct and wrong roots.

Table 1: H1. Crossover p^*, Δ (points) at $p_w = 0.95$ with 95% CI over orders, share of top-1 evidence that is an entry (self), and steal rate, by entry design. Rows marked (paper 1) are the pointer entries of Guthikonda [2026].

forum	retriever	entry	p^*	Δ at 0.95	self 0.95	steal 0.95
android	bge	ask text (paper 1)	0.47	-1.87 ± 0.65	65%	34%
android	bge	ask + thread (paper 1)	0.77	-0.04 ± 0.14	19%	3%
android	bge	generated answer	0.74	-0.19 ± 0.21	27%	10%
android	bge	ask + generated answer	>0.95	+0.14 ± 0.24	19%	6%
android	bge	generated answer + cited thread	0.89	-0.04 ± 0.21	25%	9%
android	MiniLM	ask text (paper 1)	0.67	-0.29 ± 0.24	34%	10%
android	MiniLM	ask + thread (paper 1)	>0.95	+0.25 ± 0.14	24%	3%
android	MiniLM	generated answer	>0.95	+0.15 ± 0.12	10%	2%
android	MiniLM	ask + generated answer	>0.95	+0.25 ± 0.17	15%	3%
android	MiniLM	generated answer + cited thread	0.38	-0.14 ± 0.14	9%	3%
android	BM25	ask text (paper 1)	>0.95	+0.03 ± 0.10	7%	2%
android	BM25	ask + thread (paper 1)	>0.95	+0.02 ± 0.19	19%	4%
android	BM25	generated answer	>0.95	+0.05 ± 0.07	6%	1%
android	BM25	ask + generated answer	>0.95	+0.20 ± 0.08	10%	1%
android	BM25	generated answer + cited thread	>0.95	+0.29 ± 0.11	9%	0%
unix	bge	ask text (paper 1)	0.26	-2.68 ± 0.57	56%	30%
unix	bge	ask + thread (paper 1)	0.81	-0.04 ± 0.14	11%	3%
unix	bge	generated answer	0.83	-0.08 ± 0.20	16%	6%
unix	bge	ask + generated answer	>0.95	+0.25 ± 0.15	8%	2%
unix	bge	generated answer + cited thread	>0.95	+0.22 ± 0.13	12%	3%
unix	MiniLM	ask text (paper 1)	0.63	-0.47 ± 0.24	27%	11%
unix	MiniLM	ask + thread (paper 1)	>0.95	+0.10 ± 0.16	16%	3%
unix	MiniLM	generated answer	0.84	-0.05 ± 0.12	8%	3%
unix	MiniLM	ask + generated answer	>0.95	+0.11 ± 0.13	10%	3%
unix	MiniLM	generated answer + cited thread	>0.95	+0.32 ± 0.08	6%	1%
unix	BM25	ask text (paper 1)	>0.95	+0.04 ± 0.05	2%	0%
unix	BM25	ask + thread (paper 1)	>0.95	+0.09 ± 0.05	9%	0%
unix	BM25	generated answer	0.40	-0.03 ± 0.07	5%	2%
unix	BM25	ask + generated answer	>0.95	+0.04 ± 0.07	7%	3%
unix	BM25	generated answer + cited thread	>0.95	+0.13 ± 0.04	7%	1%
webmasters	bge	ask text (paper 1)	0.14	-1.79 ± 0.28	72%	31%
webmasters	bge	ask + thread (paper 1)	0.32	-0.12 ± 0.09	24%	4%
webmasters	bge	generated answer	0.36	-0.36 ± 0.14	31%	8%
webmasters	bge	ask + generated answer	0.58	-0.15 ± 0.09	18%	5%
webmasters	bge	generated answer + cited thread	0.40	-0.32 ± 0.11	29%	9%
webmasters	MiniLM	ask text (paper 1)	0.77	-0.32 ± 0.26	42%	14%
webmasters	MiniLM	ask + thread (paper 1)	0.90	-0.04 ± 0.14	28%	6%
webmasters	MiniLM	generated answer	>0.95	+0.25 ± 0.14	11%	2%
webmasters	MiniLM	ask + generated answer	>0.95	+0.16 ± 0.19	17%	3%
webmasters	MiniLM	generated answer + cited thread	>0.95	+0.22 ± 0.11	10%	1%
webmasters	BM25	ask text (paper 1)	<0.10	-0.08 ± 0.03	7%	1%
webmasters	BM25	ask + thread (paper 1)	<0.10	-0.27 ± 0.07	17%	5%
webmasters	BM25	generated answer	>0.95	+0.11 ± 0.06	10%	2%
webmasters	BM25	ask + generated answer	>0.95	+0.16 ± 0.07	14%	2%
webmasters	BM25	generated answer + cited thread	>0.95	+0.09 ± 0.09	12%	2%
mathematica	bge	ask text (paper 1)	<0.10	-2.58 ± 0.30	65%	39%
mathematica	bge	ask + thread (paper 1)	0.70	-0.02 ± 0.06	11%	2%
mathematica	bge	generated answer	0.29	-0.46 ± 0.20	22%	11%
mathematica	bge	ask + generated answer	<0.10	-0.14 ± 0.06	10%	3%
mathematica	bge	generated answer + cited thread	0.64	-0.07 ± 0.12	17%	5%
mathematica	MiniLM	ask text (paper 1)	0.23	-1.15 ± 0.27	46%	22%
mathematica	MiniLM	ask + thread (paper 1)	<0.10	-0.49 ± 0.11	17%	7%
mathematica	MiniLM	generated answer	>0.95	+0.01 ± 0.19	12%	4%
mathematica	MiniLM	ask + generated answer	0.70	-0.14 ± 0.21	13%	6%
mathematica	MiniLM	generated answer + cited thread	>0.95	+0.13 ± 0.11	9%	2%
mathematica	BM25	ask text (paper 1)	>0.95	+0.00 ± 0.00	3%	0%
mathematica	BM25	ask + thread (paper 1)	<0.10	-0.04 ± 0.04	11%	1%
mathematica	BM25	generated answer	0.50	-0.07 ± 0.11	7%	4%
mathematica	BM25	ask + generated answer	0.52	-0.06 ± 0.09	10%	5%
mathematica	BM25	generated answer + cited thread	0.50	-0.08 ± 0.10	9%	4%

Table 2: H2. Entries by depth (summed over orders), share of last-round answers served from an entry of depth ≥ 2, attribution of wrong answers, and drift by evidence depth. Question-plus-answer entries, naive write-back.

forum	retriever	p_w	entries d1/d2/d3+	self d≥2 (last round)	wrong from d1 / d≥2	drift d0 / d1 / d2+
android	bge	0.3	6360/380/56	1.7%	5.4% / 0.4%	0.85 / 0.83 / 0.85
android	bge	0.95	13907/1744/292	4.7%	11.7% / 1.9%	0.85 / 0.82 / 0.81
android	MiniLM	0.3	6974/349/46	1.4%	4.4% / 0.4%	0.85 / 0.82 / 0.82
android	MiniLM	0.95	14319/1314/271	4.5%	8.9% / 2.0%	0.86 / 0.82 / 0.79
android	BM25	0.3	6257/167/9	0.1%	2.9% / 0.1%	0.84 / 0.80 / 0.78
android	BM25	0.95	14904/976/101	2.0%	6.6% / 0.7%	0.84 / 0.79 / 0.74
unix	bge	0.3	6940/195/16	0.3%	2.1% / 0.1%	0.86 / 0.84 / 0.81
unix	bge	0.95	15077/770/59	0.7%	5.1% / 0.3%	0.86 / 0.83 / 0.81
unix	MiniLM	0.3	7118/235/20	0.4%	2.8% / 0.1%	0.86 / 0.84 / 0.84
unix	MiniLM	0.95	14790/986/99	1.2%	6.5% / 0.7%	0.86 / 0.83 / 0.82
unix	BM25	0.3	6345/118/8	0.2%	1.9% / 0.1%	0.84 / 0.80 / 0.84
unix	BM25	0.95	15209/691/55	1.0%	4.7% / 0.3%	0.84 / 0.80 / 0.81
webmasters	bge	0.3	4868/218/18	1.2%	4.7% / 0.4%	0.87 / 0.84 / 0.83
webmasters	bge	0.95	11577/1346/252	4.9%	10.8% / 2.1%	0.87 / 0.84 / 0.83
webmasters	MiniLM	0.3	4966/241/16	0.9%	4.6% / 0.3%	0.87 / 0.84 / 0.82
webmasters	MiniLM	0.95	11619/1232/309	4.6%	9.9% / 2.5%	0.87 / 0.83 / 0.82
webmasters	BM25	0.3	4767/163/6	0.5%	3.6% / 0.2%	0.85 / 0.82 / 0.78
webmasters	BM25	0.95	11951/1098/147	2.4%	8.9% / 1.2%	0.85 / 0.81 / 0.77
mathematica	bge	0.3	4952/114/2	0.1%	2.8% / 0.0%	0.86 / 0.83 / 0.78
mathematica	bge	0.95	11945/846/40	1.0%	7.1% / 0.4%	0.86 / 0.83 / 0.82
mathematica	MiniLM	0.3	4925/195/11	0.3%	3.8% / 0.2%	0.86 / 0.82 / 0.81
mathematica	MiniLM	0.95	11675/1004/141	2.2%	8.4% / 1.2%	0.86 / 0.82 / 0.78
mathematica	BM25	0.3	4727/129/3	0.1%	2.5% / 0.0%	0.85 / 0.81 / 0.79
mathematica	BM25	0.95	12022/766/46	0.7%	6.2% / 0.4%	0.85 / 0.81 / 0.77

Table 3: H3. Observed crossover against the first-order plug-in estimate of Guthikonda [2026] and the depth-aware estimate, both measured at $p_w = 0.6$.

forum	retriever	entry	observed p^*	first-order estimate	depth-aware estimate
android	bge	generated answer	0.74	0.67	0.57
android	bge	ask + generated answer	>0.95	1.00	0.91
android	bge	generated answer + cited thread	0.89	0.93	0.75
android	MiniLM	generated answer	>0.95	1.00	1.00
android	MiniLM	ask + generated answer	>0.95	1.00	1.00
android	MiniLM	generated answer + cited thread	0.38	0.43	0.39
android	BM25	generated answer	>0.95	1.00	1.00
android	BM25	ask + generated answer	>0.95	–	–
android	BM25	generated answer + cited thread	>0.95	–	–
unix	bge	generated answer	0.83	0.90	0.81
unix	bge	ask + generated answer	>0.95	1.00	1.00
unix	bge	generated answer + cited thread	>0.95	1.00	1.00
unix	MiniLM	generated answer	0.84	0.76	0.72
unix	MiniLM	ask + generated answer	>0.95	1.00	1.00
unix	MiniLM	generated answer + cited thread	>0.95	1.00	1.00
unix	BM25	generated answer	0.40	0.87	0.84
unix	BM25	ask + generated answer	>0.95	1.00	1.00
unix	BM25	generated answer + cited thread	>0.95	1.00	1.00
webmasters	bge	generated answer	0.36	0.15	0.12
webmasters	bge	ask + generated answer	0.58	0.58	0.52
webmasters	bge	generated answer + cited thread	0.40	0.27	0.21
webmasters	MiniLM	generated answer	>0.95	1.00	1.00
webmasters	MiniLM	ask + generated answer	>0.95	1.00	1.00
webmasters	MiniLM	generated answer + cited thread	>0.95	1.00	1.00
webmasters	BM25	generated answer	>0.95	–	–
webmasters	BM25	ask + generated answer	>0.95	–	–
webmasters	BM25	generated answer + cited thread	>0.95	–	–
mathematica	bge	generated answer	0.29	0.32	0.30
mathematica	bge	ask + generated answer	<0.10	0.20	0.20
mathematica	bge	generated answer + cited thread	0.64	0.72	0.67
mathematica	MiniLM	generated answer	>0.95	0.88	0.82
mathematica	MiniLM	ask + generated answer	0.70	0.72	0.65
mathematica	MiniLM	generated answer + cited thread	>0.95	–	–
mathematica	BM25	generated answer	0.50	0.29	0.28
mathematica	BM25	ask + generated answer	0.52	0.37	0.35
mathematica	BM25	generated answer + cited thread	0.50	0.28	0.27

Table 4: H4. Δ (points, 95% CI over orders) at three p_w and the crossover, for naive write-back and three countermeasures. Question-plus-answer entries except "store with cited thread".

forum	retriever	rule	Δ at 0.10	Δ at 0.60	Δ at 0.95	p^*
android	bge	none	+0.64 ± 0.15	+0.23 ± 0.24	+0.14 ± 0.24	>0.95
android	bge	reopen demotion	+0.55 ± 0.20	+0.46 ± 0.19	+0.29 ± 0.25	>0.95
android	bge	store with cited thread	+0.81 ± 0.12	+0.20 ± 0.20	-0.04 ± 0.21	0.89
android	bge	provenance λ=0.5	+0.25 ± 0.09	+0.11 ± 0.13	+0.11 ± 0.09	>0.95
android	bge	provenance λ=1.0	+0.11 ± 0.05	+0.08 ± 0.04	+0.09 ± 0.05	>0.95
android	MiniLM	none	+0.64 ± 0.14	+0.39 ± 0.20	+0.25 ± 0.17	>0.95
android	MiniLM	reopen demotion	+0.69 ± 0.14	+0.52 ± 0.16	+0.48 ± 0.18	>0.95
android	MiniLM	store with cited thread	+0.12 ± 0.06	-0.05 ± 0.11	-0.14 ± 0.14	0.38
android	MiniLM	provenance λ=0.5	+0.09 ± 0.06	+0.01 ± 0.06	-0.03 ± 0.06	0.73
android	MiniLM	provenance λ=1.0	+0.02 ± 0.03	-0.01 ± 0.04	-0.04 ± 0.04	0.30
android	BM25	none	+0.15 ± 0.07	+0.17 ± 0.05	+0.20 ± 0.08	>0.95
android	BM25	reopen demotion	+0.14 ± 0.07	+0.18 ± 0.09	+0.21 ± 0.11	>0.95
android	BM25	store with cited thread	+0.24 ± 0.08	+0.25 ± 0.08	+0.29 ± 0.11	>0.95
android	BM25	provenance λ=0.5	+0.04 ± 0.02	+0.02 ± 0.05	+0.04 ± 0.04	>0.95
android	BM25	provenance λ=1.0	+0.00 ± 0.00	+0.00 ± 0.00	+0.00 ± 0.00	>0.95
unix	bge	none	+0.44 ± 0.08	+0.28 ± 0.12	+0.25 ± 0.15	>0.95
unix	bge	reopen demotion	+0.43 ± 0.06	+0.37 ± 0.12	+0.40 ± 0.11	>0.95
unix	bge	store with cited thread	+0.48 ± 0.07	+0.34 ± 0.09	+0.22 ± 0.13	>0.95
unix	bge	provenance λ=0.5	+0.18 ± 0.05	+0.15 ± 0.08	+0.16 ± 0.06	>0.95
unix	bge	provenance λ=1.0	+0.02 ± 0.02	+0.02 ± 0.02	+0.02 ± 0.02	>0.95
unix	MiniLM	none	+0.35 ± 0.10	+0.19 ± 0.11	+0.11 ± 0.13	>0.95
unix	MiniLM	reopen demotion	+0.39 ± 0.09	+0.40 ± 0.12	+0.29 ± 0.14	>0.95
unix	MiniLM	store with cited thread	+0.37 ± 0.08	+0.27 ± 0.07	+0.32 ± 0.08	>0.95
unix	MiniLM	provenance λ=0.5	+0.02 ± 0.03	-0.03 ± 0.04	-0.03 ± 0.07	0.26
unix	MiniLM	provenance λ=1.0	+0.01 ± 0.03	-0.01 ± 0.03	+0.01 ± 0.05	0.57
unix	BM25	none	+0.14 ± 0.09	+0.12 ± 0.10	+0.04 ± 0.07	>0.95
unix	BM25	reopen demotion	+0.15 ± 0.08	+0.09 ± 0.05	+0.09 ± 0.08	>0.95
unix	BM25	store with cited thread	+0.17 ± 0.06	+0.15 ± 0.05	+0.13 ± 0.04	>0.95
unix	BM25	provenance λ=0.5	+0.05 ± 0.04	+0.06 ± 0.03	+0.05 ± 0.04	>0.95
unix	BM25	provenance λ=1.0	+0.02 ± 0.02	+0.02 ± 0.02	+0.02 ± 0.02	>0.95
webmasters	bge	none	+0.18 ± 0.07	-0.01 ± 0.09	-0.15 ± 0.09	0.58
webmasters	bge	reopen demotion	+0.22 ± 0.09	+0.09 ± 0.07	+0.00 ± 0.13	0.88
webmasters	bge	store with cited thread	+0.14 ± 0.09	-0.16 ± 0.10	-0.32 ± 0.11	0.40
webmasters	bge	provenance λ=0.5	+0.02 ± 0.02	-0.04 ± 0.04	-0.06 ± 0.03	0.23
webmasters	bge	provenance λ=1.0	+0.00 ± 0.00	+0.00 ± 0.00	+0.00 ± 0.00	>0.95
webmasters	MiniLM	none	+0.34 ± 0.15	+0.29 ± 0.21	+0.16 ± 0.19	>0.95
webmasters	MiniLM	reopen demotion	+0.30 ± 0.13	+0.32 ± 0.19	+0.26 ± 0.12	>0.95
webmasters	MiniLM	store with cited thread	+0.24 ± 0.09	+0.20 ± 0.14	+0.22 ± 0.11	>0.95
webmasters	MiniLM	provenance λ=0.5	+0.11 ± 0.06	+0.10 ± 0.06	+0.10 ± 0.07	>0.95
webmasters	MiniLM	provenance λ=1.0	+0.00 ± 0.00	+0.00 ± 0.00	+0.00 ± 0.00	>0.95
webmasters	BM25	none	+0.14 ± 0.05	+0.16 ± 0.07	+0.16 ± 0.07	>0.95
webmasters	BM25	reopen demotion	+0.14 ± 0.07	+0.14 ± 0.06	+0.15 ± 0.07	>0.95
webmasters	BM25	store with cited thread	+0.09 ± 0.02	+0.07 ± 0.06	+0.09 ± 0.09	>0.95
webmasters	BM25	provenance λ=0.5	+0.00 ± 0.00	+0.01 ± 0.02	+0.01 ± 0.02	>0.95
webmasters	BM25	provenance λ=1.0	+0.00 ± 0.00	+0.00 ± 0.00	+0.00 ± 0.00	>0.95
mathematica	bge	none	-0.00 ± 0.09	-0.06 ± 0.04	-0.14 ± 0.06	<0.10
mathematica	bge	reopen demotion	+0.02 ± 0.07	-0.01 ± 0.06	-0.06 ± 0.07	0.17
mathematica	bge	store with cited thread	+0.15 ± 0.07	+0.03 ± 0.12	-0.07 ± 0.12	0.64
mathematica	bge	provenance λ=0.5	+0.00 ± 0.00	+0.01 ± 0.02	+0.04 ± 0.03	>0.95
mathematica	bge	provenance λ=1.0	+0.00 ± 0.00	+0.01 ± 0.02	+0.04 ± 0.03	>0.95
mathematica	MiniLM	none	+0.33 ± 0.11	+0.05 ± 0.21	-0.14 ± 0.21	0.70
mathematica	MiniLM	reopen demotion	+0.30 ± 0.12	+0.18 ± 0.11	+0.14 ± 0.19	>0.95
mathematica	MiniLM	store with cited thread	+0.18 ± 0.08	+0.17 ± 0.12	+0.13 ± 0.11	>0.95
mathematica	MiniLM	provenance λ=0.5	+0.07 ± 0.04	+0.02 ± 0.08	-0.04 ± 0.08	0.29
mathematica	MiniLM	provenance λ=1.0	+0.04 ± 0.03	+0.04 ± 0.04	+0.04 ± 0.04	>0.95
mathematica	BM25	none	+0.08 ± 0.09	-0.06 ± 0.12	-0.06 ± 0.09	0.52
mathematica	BM25	reopen demotion	+0.10 ± 0.11	+0.01 ± 0.12	+0.05 ± 0.11	0.84
mathematica	BM25	store with cited thread	+0.04 ± 0.10	-0.04 ± 0.08	-0.08 ± 0.10	0.50
mathematica	BM25	provenance λ=0.5	+0.03 ± 0.05	+0.05 ± 0.03	+0.08 ± 0.04	>0.95
mathematica	BM25	provenance λ=1.0	+0.00 ± 0.00	+0.00 ± 0.00	+0.00 ± 0.00	>0.95

Table 5: H1 on android with the open-weight generator (compare the android rows of Table 1).

forum	retriever	entry	p^*	Δ at 0.95	self 0.95	steal 0.95
android	bge	ask text (paper 1)	0.47	-1.87 ± 0.65	65%	34%
android	bge	ask + thread (paper 1)	0.77	-0.04 ± 0.14	19%	3%
android	bge	generated answer	0.61	-0.21 ± 0.22	25%	9%
android	bge	ask + generated answer	>0.95	+0.08 ± 0.19	20%	7%
android	bge	generated answer + cited thread	0.68	-0.11 ± 0.13	23%	7%
android	MiniLM	ask text (paper 1)	0.67	-0.29 ± 0.24	34%	10%
android	MiniLM	ask + thread (paper 1)	>0.95	+0.25 ± 0.14	24%	3%
android	MiniLM	generated answer	0.93	-0.02 ± 0.11	10%	2%
android	MiniLM	ask + generated answer	>0.95	+0.04 ± 0.11	16%	3%
android	MiniLM	generated answer + cited thread	0.54	-0.05 ± 0.09	8%	2%
android	BM25	ask text (paper 1)	>0.95	+0.03 ± 0.10	7%	2%
android	BM25	ask + thread (paper 1)	>0.95	+0.02 ± 0.19	19%	4%
android	BM25	generated answer	>0.95	+0.03 ± 0.06	5%	1%
android	BM25	ask + generated answer	>0.95	+0.18 ± 0.06	10%	1%
android	BM25	generated answer + cited thread	>0.95	+0.20 ± 0.11	9%	1%

Table 6: H2 on android with the open-weight generator (compare the android rows of Table 2).

forum	retriever	p_w	entries d1/d2/d3+	self d≥2 (last round)	wrong from d1 / d≥2	drift d0 / d1 / d2+
android	bge	0.3	6312/425/64	1.6%	5.4% / 0.4%	0.85 / 0.85 / 0.87
android	bge	0.95	13770/1823/355	4.9%	12.0% / 2.3%	0.85 / 0.84 / 0.84
android	MiniLM	0.3	6943/370/33	1.0%	4.7% / 0.4%	0.86 / 0.84 / 0.83
android	MiniLM	0.95	14211/1398/298	4.3%	9.5% / 2.2%	0.86 / 0.84 / 0.83
android	BM25	0.3	6242/180/14	0.4%	2.8% / 0.1%	0.84 / 0.82 / 0.81
android	BM25	0.95	14934/969/80	1.6%	6.5% / 0.5%	0.84 / 0.82 / 0.80

Table 7: Faithfulness audit: share of answers judged to state the root thread's solution, by evidence depth and evidence label.

evidence depth	label	n	gpt-5.4-mini judge			open-weight judge		
			yes	partly	no	yes	partly	no
0 (thread)	correct	10	80%	0%	20%	80%	0%	20%
0 (thread)	wrong	90	67%	22%	11%	86%	2%	12%
1	correct	5	100%	0%	0%	100%	0%	0%
1	wrong	95	65%	21%	14%	75%	6%	19%
2+	correct	5	80%	20%	0%	100%	0%	0%
2+	wrong	95	55%	23%	22%	67%	8%	24%